Node Canvas — Design Direction & Working Product Vision

Living design record • Working decisions and open questions as of August 12, 2026

Purpose

This document captures the design direction discussed so far for Node Canvas: a canvas-based environment where nodes hold information, writing, ideas, entities, and reasoning structures; wires and references show relationships; and the same project can support creative writing, academic work, research, study, note-taking, and other knowledge-heavy workflows. This is a working design record, not a final product specification. The goal is to preserve decisions and ideas so later design work can build on them without losing the reasoning behind them.

1. Core product philosophy

Node Canvas should feel like a place to think and build, rather than a database that requires the user to know the final structure in advance. The user should be able to begin with almost nothing, throw down a few ideas, and gradually develop them into structured knowledge and finished writing.

- The canvas is where the user thinks and builds.
- Nodes are the things the user is thinking about, writing about, researching, or using to reason.
- Connections preserve relationships between nodes, but ordinary connections should stay simple by default.
- Documents and nodes should be part of the same underlying project rather than separate systems.
- Structure should emerge from work instead of being required before the work begins.
- The system should have a low floor and a high ceiling: a node can be useful with almost no information, then become more sophisticated through fields, references, linked content, and relationships.
- The interface should feel somewhat like building with Lego: each node type is a recognizable piece with a purpose, but the pieces can be combined freely into a larger project.
- The system should never force the user to decide how organized they need to be before they are ready to organize.

2. Why the current port-heavy approach is being reconsidered

The current implementation has explicit typed ports such as People, Setting, POV, Claims, Sources, and similar connection points. The current direction is to remove or greatly reduce visible ports for ordinary nodes because they can make Node Canvas feel like a visual programming environment and can force structure that is unnecessary for many relationships.

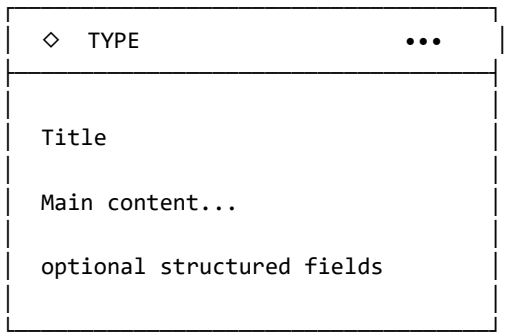
A character connected to a chapter does not need to connect through a People port. A location connected to a chapter does not need a Setting port. The connection itself can communicate the relationship, with optional semantics added when useful.

New working rule: ordinary nodes should have simple connections by default. Typed relationships, fields, and logic structures remain available when they provide genuine value, but they should not be mandatory infrastructure for normal use.

3. The node model

The preferred architecture is a small set of reusable node primitives plus templates and customizable fields. A node should be more than a fixed card type, but it should also not become an entirely generic blank box.

Universal node anatomy



The exact visual design is still open, but the consistent idea is that the node body is the actual working object. Nodes should not look like collections of exposed ports or programming components.

4. Custom fields

A major part of the design should be the ability to add fields to a node when a user wants more structure. Built-in types provide useful defaults, but fields should not be mandatory or fixed forever.

Potential field types

- Short text
- Long text / rich text
- Number
- Date / time
- Checkbox / boolean
- Dropdown / single select
- Multi-select / tags
- Color
- Image / media
- URL
- Rating
- Node reference
- Node list / collection
- Formula or calculated value (future possibility)

A field can reference another node. For example, a Character can have a Mentor field pointing to a Thalvior node, or a Location can have a related Event field. These references should not require visible ports.

Example: Character

```
CHARACTER
Durvain Ironscribe

Dwarven scholar.

Role
Dwarven scholar / scribe

Age
3,284

Goals
Understand what happened to Deepvault

Fears
Failing those who depended on him

Voice
Formal, restrained, occasionally dry

Arc
Scholar → Survivor → Leader
```

The same Character node should also be valid when it contains only a name and one sentence. Fields can be added later.

5. Node families

Thinking / capture nodes: Note, Idea, Question, Quote, Reminder

Things / knowledge nodes: Person, Place, Concept, Object, Event, Theme, Source

Writing / work nodes: Document, Chapter, Scene, Outline, Argument, Study

Reasoning / flow nodes: Decision, Condition, Sequence, AND, OR, NOT, Compare, Merge, Split, Transform, Filter

Reference / navigation nodes: Compact Link / Reference node

6. Core nodes discussed so far

Node	Purpose	Important behavior
Note	Fast capture of information or thoughts	Minimal friction; no mandatory title or fields
Idea	Potential concept to develop	Visually distinct from Note, but similarly lightweight
Question	An unresolved question	Can connect to notes, research, ideas, arguments, etc.; no mandatory answer port
Document	General long-form writing	Real rich-text writing area; suitable for essays, sermon notes, research papers, etc.

Person	Universal individual/entity foundation	Can support Character, Historical Figure, Scholar, Patient, etc. through templates or fields
Character	Fiction-focused person node	Useful built-in fields such as role, goals, fears, relationships, voice, arc; all optional
Place / Location	Persistent location or setting	Can hold description, atmosphere, history, images, notes, related scenes/events
Concept	Abstract idea	Useful for themes, technical ideas, theological concepts, scientific concepts, etc.
Object / Item	Persistent named thing	Useful for story items, artifacts, instruments, objects of study, etc.
Event	An occurrence in time	Can be used in stories, history, research, study, nursing scenarios, etc.
Theme	A recurring or important idea	Can connect to scenes, chapters, characters, evidence, claims, etc.
Source	External source/reference	Author, title, publication, date, URL/DOI, citation data, notes
Quote	Captured quotation	Text, speaker/author, source, page/verse, context, notes
Definition	Meaning of a term	Useful across academics, languages, note-taking, CS, nursing, etc.
Claim	Proposition to be supported or tested	Can connect to evidence, sources, counterarguments, and conclusions
Evidence	Support for a claim	Source, location, what it demonstrates, strength/confidence, limitations
Argument	Structured reasoning	Can contain claims, premises, evidence, counterarguments, and conclusions
Counterargument	Opposing position	Can preserve the objection and the response
Passage	A bounded piece of source text	Useful for Bible study, literature, academic analysis, etc.
Word	Word-level text object	Can hold lemma, part of speech, morphology, translation, occurrences, notes
Translation	Language comparison / rendering	Original, transliteration, literal, formal/interpretive rendering, notes
Syntax / Language Analysis	Structure of language	Can represent phrases, clauses, dependencies, grammatical relationships
Domain template	Discipline-specific knowledge structure	Prefer templates built from general node primitives rather than dozens of hard-coded node classes

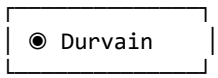
7. Node visual scale: clarified direction

The earlier idea of three explicit presentation scales (compact / standard / expanded) has been revised. The preferred direction is a standard node that simply grows as the user types or adds fields. The user can manually resize the node to the dimensions they want. A separate compact form is specifically for the standalone Link / Reference node rather than for turning every normal node into a smaller mode.

Normal nodes

A Note, Character, Scene, Chapter, etc. should have a standard visual form. As content grows, the node grows naturally within reasonable layout rules. The user can resize it manually. There does not need to be a separate expanded node state.

Compact Link / Reference node



This is a real standalone node placed on the canvas. It points to another full node while remaining visually small. It can be connected to the standard node, color-coded, moved, grouped, and used in large numbers. It is intended to make relationships visible without turning important nodes into giant spiderwebs. The compact Link node should be thought of as a small Lego tile or reference piece, not as a pill inside the Chapter or Scene.

8. Context strips vs. link nodes

A previous proposal suggested a context strip inside a Chapter or Scene containing pills such as Durvain, Thalvior, Myrelia, Deepvault, and River. That is not the preferred approach. Instead, those references should remain actual compact Link nodes outside the primary writing node. A Chapter or Scene can therefore be surrounded by many small reference nodes that show what belongs to it or relates to it. A contextual relationship summary may still be useful as a small optional UI element, but it should not replace the actual Link nodes.

9. Smart linking inside writing and notes

Smart linking is strongly desired and should work across the whole application, not just in fiction writing. When the user types the name of an existing node, the editor can recognize it and offer to link the text to that node. Example: a note containing “Photosynthesis requires light energy and produces glucose” could recognize existing nodes named Photosynthesis, Light Energy, and Glucose and allow those occurrences to become references. The same behavior could work for Character names, Locations, Themes, Bible terms, authors, sources, technical terms, diseases, etc.

The system should favor suggestions or confirmation over blindly creating permanent relationships every time a matching word is typed. Trust is important.

Hover preview

Hovering over a linked node reference in text should provide a compact preview without leaving the current work. A preview might show the node type, summary, key fields, and how many related notes/scenes/etc. exist.

10. Chapter and Scene design

Chapter and Scene are the most important writing-specific nodes for the current design. Both should contain real writing and remain usable before their final structure is known.

Chapter

```
CHAPTER 1
The Dwarven Scribe

Durvain returned to the library...

He had spent three hundred years studying...

The river moved beneath the stone...
```

A Chapter should be able to function almost like a Google Docs document. The user can write the entire chapter directly without creating Scenes first. Later, sections can be extracted or organized into Scene nodes.

Scene

```
SCENE
The Return to Deepvault

Deepvault Library
Durvain · Thalvior

Durvain stepped through the ancient archway and heard the river before he saw it...
```

A Scene is a focused writing unit, but it should not require a specific planning schema to begin. It can exist as a small idea or become a full-length scene of prose.

11. Node-backed document blocks

A major desired capability is a node-backed block: a Scene or other node can appear inside a Chapter document as a visually identifiable block while remaining a real node elsewhere on the canvas. The Chapter document and the Scene node should not be two independent copies of the same text. They should be two views of the same underlying content.

```
CHAPTER 1

[Scene: The Arrival]
Durvain crossed the threshold...

[Scene: The Library]
The river moved beneath the floor...

[Scene: The Conversation]
Thalvior waited in the shadows...
```

The Chapter therefore visually shows which Scene nodes it contains, while the canvas can separately show the relationships among the Chapter, Scenes, Characters, Locations, Themes, and other nodes.

12. Split, combine, extract, and merge

The current split/combine behavior should be redesigned around writing workflows. These actions should be easy to discover and should preserve information rather than forcing destructive restructuring.

Split: Take selected or bounded text and create a new node-backed section, such as a Scene.

Combine: Merge multiple Scene or writing sections into one continuous piece of text while preserving the relevant node data and relationships.

Extract: Create a new node from selected content without removing the source text. Example: select a character description inside a Chapter and extract a Character node.

Embed / Insert: Place an existing Scene or node-backed block inside a Chapter or other document.

Detach / Remove: Remove a Scene from a Chapter without deleting the Scene node itself.

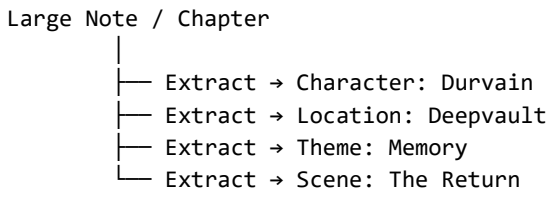
Move: Move a Scene from one Chapter to another while preserving the Scene node.

Replace / Alternate: Allow an alternative version of a scene or passage to exist as a separate node without destroying the original.

These operations should have visible, understandable controls near the writing experience rather than being hidden exclusively in obscure menus. Hover descriptions can teach the user what each action does.

13. Extraction as a core workflow

A project may begin with only a few broad nodes and gradually become structured. A large note or Chapter can contain enough information to extract a Character, Location, Theme, Event, or Scene without deleting the original information.



The original content remains intact, and the new node becomes a more structured representation connected to the source. This supports the desired “start simple, structure later” workflow.

14. Logic and flow nodes

Logic nodes are desired, but they should be optional reasoning tools rather than the default way that information is connected. Their purpose is to show flow, progression, conditions, and transformations where a plain relationship is not enough.

Sequence: Shows progression from one thought/event/state to the next. Useful for story progression, research processes, sermon structure, study workflows, and CS algorithms.

Decision: Creates a branch based on a choice or condition.

Condition: Represents IF / THEN / ELSE reasoning.

AND: Requires multiple inputs/conditions.

OR: Allows one or more alternatives.

NOT: Negates a condition or statement.

Compare: Places two or more things into a deliberate comparison.

Merge: Combines several ideas or paths into one.

Split: Branches one input into multiple paths.

Transform: Represents an input becoming something else, such as notes → outline → draft.

Filter: Selects a subset of connected information based on conditions or attributes.

Container / Group: Allows many pieces to live under a larger conceptual or project unit while still allowing extraction of independent nodes.

Sequence is especially important because it can describe story progression, academic reasoning, sermon structure, scientific/research workflows, and computational processes without needing separate systems for each discipline.

15. Story-writing workflow example

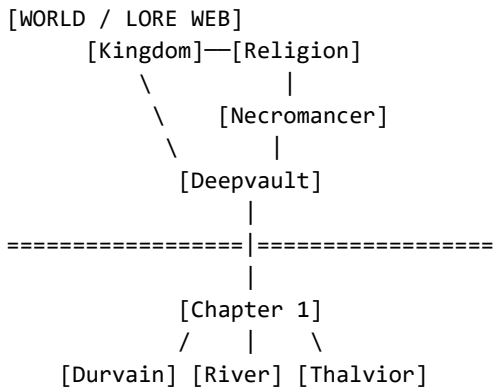
1. Start a new project and write a broad novel/story idea in a Note or Document.
2. Add scattered details without deciding their final type: Dwarven scholar, deep mountain library, river through the mountain, possible necromancer antagonist, biblical themes, possible trilogy structure, etc.
3. Gradually promote or convert useful ideas into structured nodes such as Character, Location, Theme, Event, Object, or Question.
4. Create an early Chapter or Scene when enough material exists, even if the outline is incomplete.
5. Create Character nodes such as Durvain, Thalvior, Thalarian, Myrelia, and other important characters so their names and information remain persistent.
6. Create Location, Object, Theme, and other nodes for details the writer does not want to forget.
7. Connect those nodes to the Chapter or Scene using ordinary connections and/or compact Link nodes rather than explicit ports.
8. Write the Chapter directly like a normal document, while being able to see nearby reference nodes for character names, locations, objects, and ideas.
9. Create alternative ideas or alternate versions as side nodes without destroying the main writing.
10. Split sections into Scenes when useful, or bring existing Scenes into the Chapter as node-backed document blocks.
11. Move, combine, split, or extract sections as the story structure becomes clearer.
12. Repeat the process for the next Chapter, gradually turning the canvas into a connected representation of the story world and manuscript.

16. Writing should not force structure

A user should be able to do everything in a few large nodes if that is how they naturally work. A project should not require separate Characters, Scenes, Locations, Themes, etc. from the start. Those can be extracted later. Conversely, a highly structured user should be able to build a large web of specific nodes from the beginning. This “few nodes or many nodes” flexibility is a major product goal.

17. Reference nodes and canvas organization

A large project may contain dense worldbuilding webs on one part of the canvas and a sequence of Chapter nodes on another. Compact Link nodes are intended to let the user place many small references around an important writing node without making the main node huge or forcing long wires across the entire canvas.



Compact references can be color-coded. The colors should help the user read the structure without becoming rigid discipline-specific semantics.

18. Color and shape philosophy

Shape should primarily communicate what kind of thing a node is. Color should remain customizable and should not permanently mean “fiction,” “nursing,” or “computer science.”

Shape examples: Circle or rounded shape for entities or concepts; document-like form for writing nodes; diamond for decisions; gate or connector shape for logic; compact card/tile for references. Exact shapes remain open.

Color: User-controlled or template-controlled semantic emphasis. A novelist might use one color for Characters, another for Locations, etc.; a nursing student might use colors for Diseases, Symptoms, Treatment, Diagnostics.

Consistency: All nodes should feel like parts of the same visual construction system even when their purpose differs.

19. Important distinction: node type vs. template

The system should not hard-code a separate class for every imaginable discipline-specific object. A smaller set of foundational concepts can support templates. For example: Person → Character, Historical Figure, Scholar, Patient; Concept → Theme, Disease, Technical Concept; Document → Chapter, Scene, Essay, Sermon, Study. Templates can provide suggested fields without turning those fields into mandatory requirements.

20. Example cross-domain usage

Creative writing: Character → Scene → Chapter → Theme → Location → Object → Event

Bible study / exegesis: Passage → Word → Translation → Observation/Question → Claim → Evidence → Commentary/Source → Application

Nursing study: Disease → Symptoms → Risk Factors → Diagnostics → Treatment → Complications → Care Plan

Computer science: Concept → Algorithm → Function → Data Structure → Bug → Test → Result

Academic research: Question → Source → Evidence → Claim → Counterargument → Argument → Paper

Sermon preparation: Text → Observation → Interpretation → Claim → Illustration → Application → Sermon Document

21. Existing program strengths to preserve

The uploaded project already contains several useful foundations that should be preserved and evolved rather than discarded. The current system includes a generic node schema with arbitrary data fields, a central node registry, relationship and typed data-wire concepts, collapsible assemblies, a rich-text editor, document structures with writing blocks, embedded node content, extraction/forking behavior, node-specific renderers, and existing writing-oriented modes. The redesign should take advantage of this existing foundation while reducing unnecessary visual complexity.

22. Current design direction at a glance

Decision / direction	Current working position
Ports	Remove or greatly reduce explicit visible ports for ordinary nodes. Keep specialized/typed logic available when useful.
Custom fields	Yes. Fields are a core feature and can be added, removed, reordered, and customized.
Normal node growth	Standard node grows as content or fields are added; user can resize it. No separate expanded state is required.
Compact links	Yes. A standalone compact Link/Reference node points to another node and stays small on the canvas.
Context strip	Not as a replacement for Link nodes. Optional supporting context may exist, but references should remain visible as actual nodes.
Smart linking	Yes. Works throughout notes, documents, research, Bible study, etc.; should usually suggest/confirm rather than auto-link blindly.
Hover preview	Yes. Hovering a linked reference can show a small preview without leaving the current context.
Chapter	Can be written directly like a Google Doc; does not require Scenes.
Scene	Can begin small and grow into full prose; can exist independently or be embedded as a node-backed block in a Chapter.
Split/Combine	Should be rebuilt around writing operations and remain non-destructive where possible.
Extract	Core operation: create a structured node from existing content while keeping the original.
Logic nodes	Yes. Optional tools for flow, conditions, decisions, sequences, transformations, etc.

Sequence node	Particularly promising for narrative, academic, sermon, research, and CS flows.
Visual philosophy	Lego-like construction system: recognizable pieces, compatible interactions, low floor, high ceiling.
Overall goal	Allow a project to remain simple with a few large nodes or become deeply structured with many connected nodes.

A **Template** in Node Canvas should be something the user creates from their own way of working, rather than something we only ship with the program.

For example, you could build a little story-planning structure:

```
        [Character]
            |
            |
[Location] —— [Scene] —— [Theme]
            |
          [Idea]
```

You select all five nodes and choose:

Create Template

Then:

Name:

Now, from the Add Node menu, you could have:

Nodes

- Note
- Idea
- Character
- Scene
- Location
- ...

Templates

- Story Scene Starter
- Bible Passage Study
- Research Argument
- Nursing Disease Study
- My Custom Project Structure

Selecting would place a new copy of that entire structure onto the canvas.

The important part is that the new copy should be independent. Its nodes get new IDs, while preserving the same internal relationships, fields, colors, shapes, and layout.

I would give templates two creation modes

Create Template from Selection

This is exactly what you described.

Select several nodes → → name it → optionally add a description/icon.

The selected layout becomes the template.

Save Node as Template

For a particularly useful customized node, you could also right-click:

That would preserve its fields, appearance, and configuration.

So you could make something like:

Detailed Character

with:

- Name
- Role
- Goals
- Fears
- Personality
- Relationships
- Arc
- Notes

and use it whenever you create a new character.

Templates should also preserve more than just the fields

I think a saved template should remember:

- node types
- custom fields
- field order
- colors
- shapes
- sizes
- internal connections

- relationship labels
- compact/standard presentation where appropriate
- embedded node-backed structures
- perhaps default text/placeholders

So a template is essentially a **reusable Lego construction**.

That gives us another very useful property:

> **Node Canvas can start simple, and the user gradually teaches it how they personally work.**

You could begin with the built-in nodes.

After six months, your Node Canvas might have:

```
Novel Character

Novel Location

Chapter Planning

Bible Passage Study

Greek Word Study

Sermon Outline

Research Argument
```

I would also make templates available directly in the same creation menu we discussed earlier, but visually separated:

Create Node

Create Template

That way a first-time user sees the built-in node types immediately, while experienced users eventually get a personalized construction kit.

maybe for the add node and template menu should be separate under adding a normal node, so a separate menu that you start building yourself..

24. Working product statement

Node Canvas is envisioned as a flexible project environment where ideas, notes, entities, research, reasoning, and long-form writing live in one connected space. The user can begin with a single thought and build outward. Nodes can remain simple or become structured through custom fields. Writing can live directly in nodes. Documents can contain node-backed sections. Compact reference nodes can show relationships without overwhelming the canvas. Smart linking can connect text to the project graph. Logic

and flow nodes can provide deliberate structure when needed. The system should feel like building a project out of understandable pieces while preserving the freedom to work naturally before the final structure is known.